Utilisation du SPI pour communication avec des périphériques externes

Réalisé par:

1)Insaf Fares
2)Hidri Mohamed Aziz
3)Salim Belkhir

2 ATEL 2

Agenda

- Introduction
- Fonctionnement de SPI
- Connexion et Configuration SPI
- Avantages de l’utilisation du SPI
- Inconvénients de l’utilisation du SPI
- Exemples d’utilisation du SPI dans les microcontrôleurs
- Comparaison des protocoles SPI, I2C et UART
- Conclusion

life.augmented



Introduction

Introduction

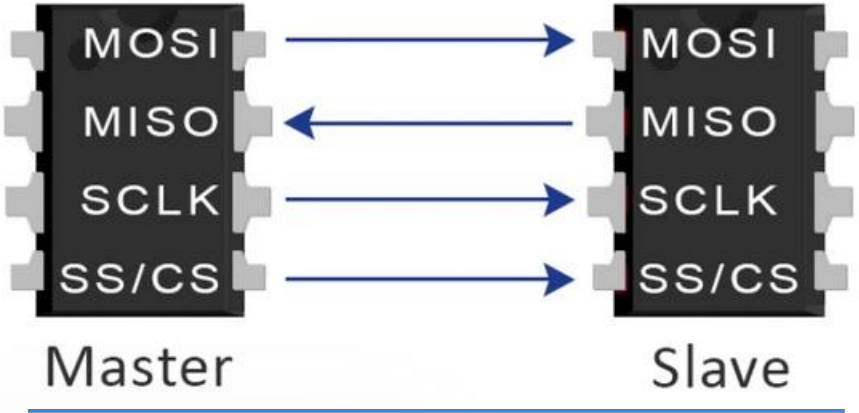
L'interface périphérique série (SPI) est une méthode de communication créée par Motorola dans les années 1980 pour permettre l'échange de données entre un microcontrôleur et divers périphériques. Bien qu'elle n'ait jamais été formalisée en tant que norme officielle, elle a rapidement été adoptée par d'autres entreprises, devenant ainsi une norme de facto dans l'industrie électronique.





Connection et configuration

MISO et MOSI du SPI



SCLK (serial clock) :
Horloge du bus (produite par le maître)

MOSI (Master Out Slave In) :
Données du maître vers l'esclave actif

MISO (Master In Slave Out) :
Données de l'esclave actif vers le maître

SSn (Slave Select n) :
Sélection de l'esclave n à destination de la transmission

Les lignes MOSI et MISO du SPI jouent un rôle clé dans le protocole de communication. Le maître envoie des données séquentiellement via la ligne MOSI, tandis que chaque esclave les reçoit et renvoie ses propres données sur la ligne MISO. La transmission s'effectue bit par bit, en commençant par le bit le plus significatif, et est synchronisée avec les impulsions d'horloge.

Connexions SPI

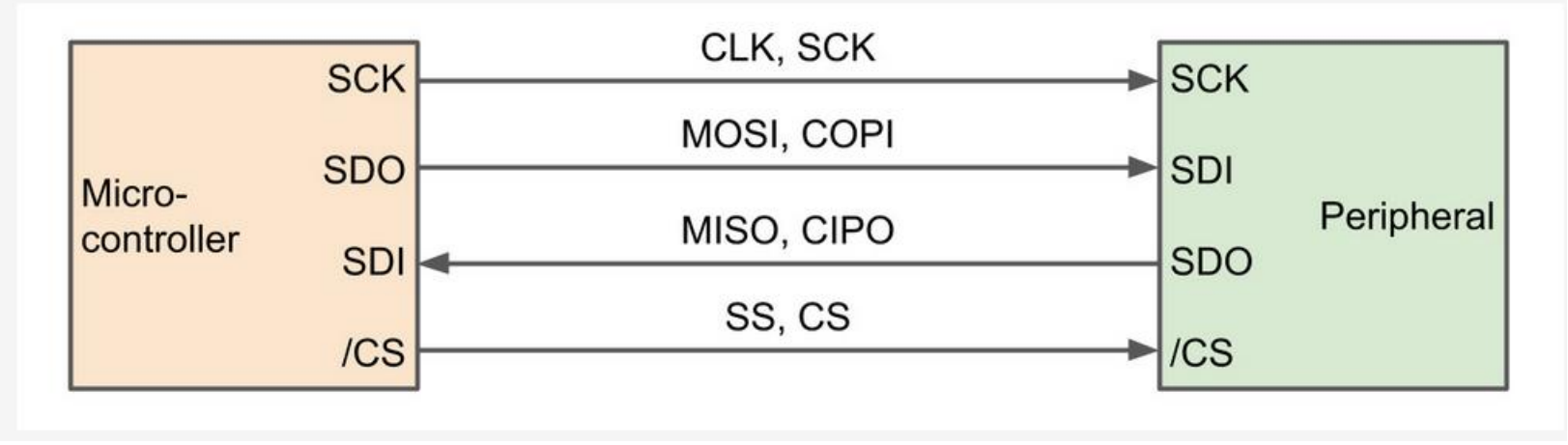
Le SPI (Serial Peripheral Interface) peut être configuré de différentes manières. La configuration la plus courante est la connexion **point à point**.
Comme le SPI est une norme non officielle, les noms des lignes peuvent varier :

- **Horloge** : clock (CLK), serial clock (SCK)
- **Données sortantes du contrôleur** : serial data out (SDO), master out slave in (MOSI), controller out peripheral in (COPI)
(La broche SDO du contrôleur doit être connectée à la broche SDI du périphérique.)
- **Données entrantes du contrôleur** : serial data in (SDI), master in slave out (MISO), controller in peripheral out (CIPO)
(La broche SDI du contrôleur doit être connectée à la broche SDO du périphérique.)
- **Sélection** : chip select (CS), slave select (SS)

Les types de connections

Connexion SPI point à point

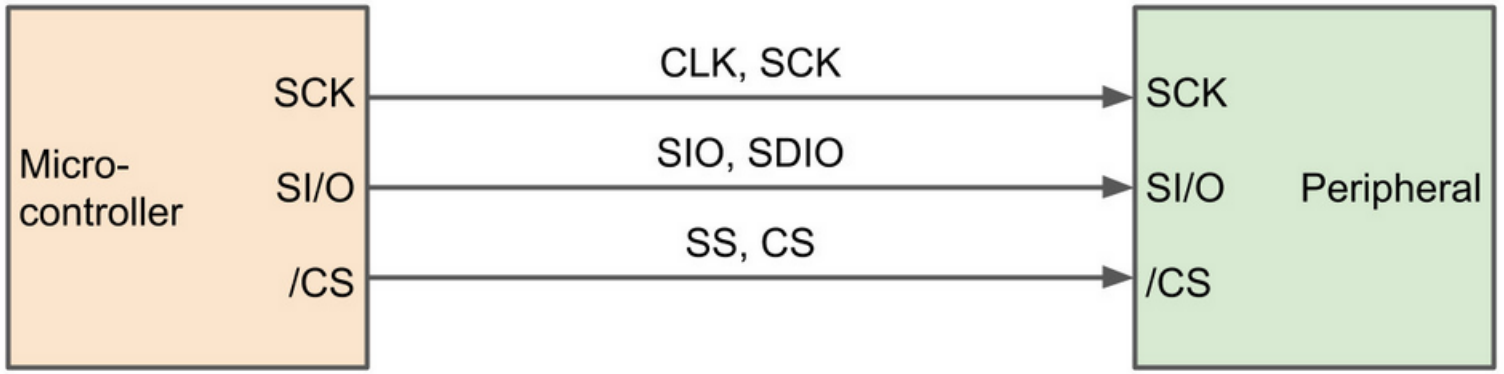
Par défaut, le SPI fonctionne en **full-duplex**, ce qui signifie qu'il peut transmettre et recevoir des données simultanément. Cependant, certains appareils ne prennent en charge que le mode **half-duplex** pour réduire le nombre de broches utilisées. Dans ce cas, la ligne de données partagée peut être appelée **serial in/out** (SI/O, SIO) ou **serial data in/out** (SDIO). Pour les composants STM32, cette ligne doit être connectée à la broche SDO (ou MOSI)



Les types de connections

Connexion SPI
en half-duplex

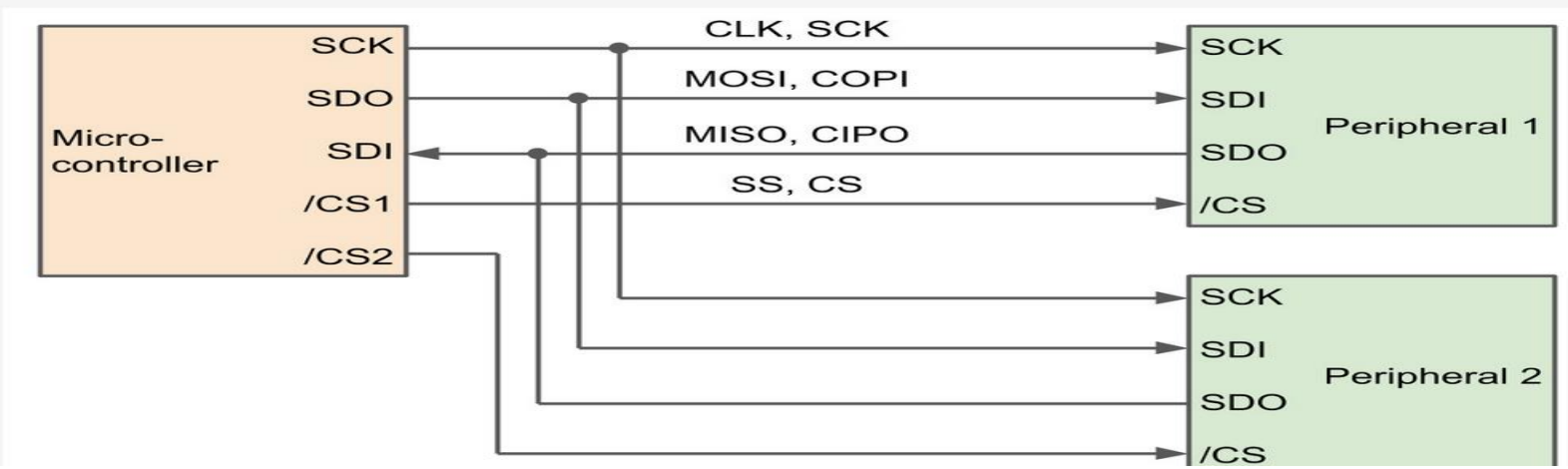
Le SPI fonctionne également comme un **bus**, permettant à un contrôleur de communiquer avec plusieurs périphériques. Toutes les lignes sont partagées sauf la ligne CS, qui doit être dédiée à chaque périphérique. Chaque périphérique connecté nécessite une broche CS distincte sur le contrôleur. Les lignes CS sont **actives à l'état bas** : elles sont à l'état haut au repos et passent à l'état bas pour sélectionner un périphérique.



Les types de connections

Connexion SPI en bus

Dans certains cas, les périphériques utilisent une configuration **en chaîne** (daisy chain), ce qui est courant pour des dispositifs comme les pilotes de LED. La broche SDO du premier périphérique est connectée à la broche SDI du suivant, et ainsi de suite. La broche SDO du dernier périphérique est ensuite reliée au contrôleur.



Modes SPI

SPI Modes

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Le SPI propose **quatre modes** qui déterminent comment les données sont synchronisées avec l'horloge :

•**CPOL** (polarité de l'horloge) :

- 0 : l'horloge est à l'état bas au repos.
- 1 : l'horloge est à l'état haut au repos.

•**CPHA** (phase de l'horloge) :

- 0 : les bits sont échantillonnés sur le front montant.
- 1 : les bits sont échantillonnés sur le front descendant.



- Fonctionnement de SPI

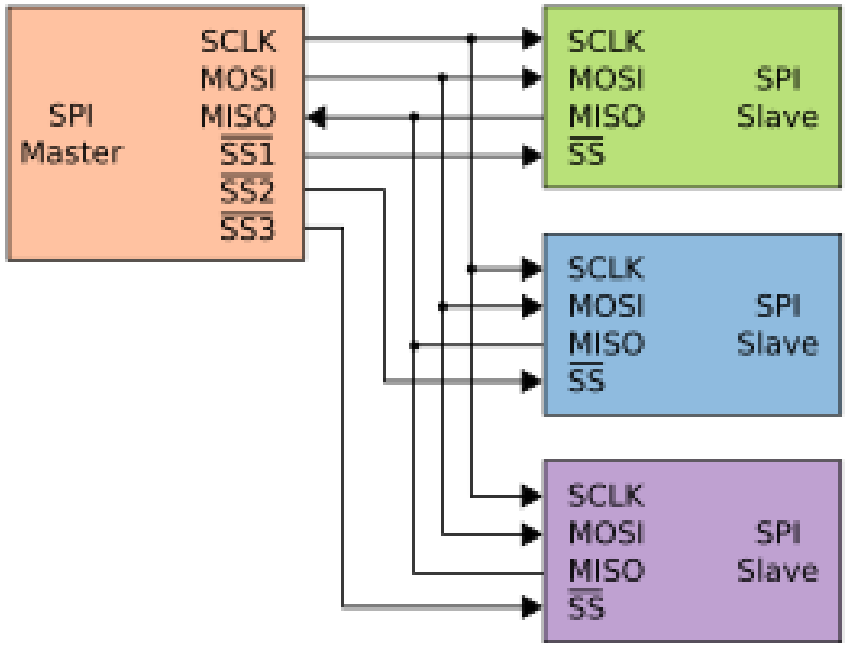
Fonctionnement

Une transmission SPI classique implique une communication simultanée entre un maître et un esclave :

•Le maître génère le signal d'horloge et choisit l'esclave avec lequel il souhaite communiquer via le signal SS.

•L'esclave répond aux requêtes émises par le maître.

À chaque impulsion d'horloge, un bit est échangé entre le maître et l'esclave. Après huit impulsions, un octet est transmis dans chaque direction. La fréquence de l'horloge est ajustée en fonction des spécifications des périphériques impliqués.





Comparaison des protocoles UART, SPI et I2C

Comparaison des protocoles UART, SPI et I2C

Caractéristiques	UART	SPI	I2C
Type de protocole	Asynchrone	Synchrone	Synchrone
Nombre de lignes	2 (TX, RX)	4 (MOSI, MISO, SCLK, SS)	2 (SDA, SCL)
Vitesse de transmission	Variable, débit fixe	Haute	Variable, débit fixe
Communication duplex	Simplex, semi-duplex	Full-duplex	Bidirectionnelle
Nombre de périphériques	2 (maître et esclave)	Plusieurs	Jusqu'à 127
Complexité	Simple	Moyenne	Simple
Utilisation des broches	Moins	Plus	Moins

life.augmented

Avantages du SPI :

Rapide et full-duplex : Permet l’envoi et la réception simultanés avec des taux de transfert élevés.

Simple et flexible : Facile à configurer avec peu d’overhead et adaptable via les modes CPOL/CPHA.

Support étendu : Compatible avec de nombreux périphériques et bien géré sur STM32, notamment via le DMA.

life.augmented

Inconvénients du SPI

Limité en distance et broches : Nécessite une ligne CS par esclave, peu adapté aux longues distances.

Pas de gestion intégré : Absence de contrôle d’erreurs ou d’arbitrage, à gérer en logiciel.

Complexité croissante : Plus difficile à gérer avec plusieurs esclaves ou à très haute fréquence.

life.augmented



- Exemples d’utilisation du SPI dans les microcontrôleurs

Mode SPI DMA sur STM32 : Fonctions HAL

Pour la transmission

```
HAL_SPI_Transmit_DMA(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

Après avoir appelé cette fonction, le périphérique SPI commencera à envoyer les octets de données présents dans le tampon, un par un, jusqu'à ce que la transmission soit terminée (en mode DMA).
Une fois la transmission terminée, la fonction ci-dessous sera appelée et exécutée.
Si vous souhaitez effectuer une action à la fin de la transmission, ajoutez-la dans votre code source de l'application (fichier main.c)

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
 { // Transmission terminée .. Faites quelque chose ... }
```

Mode SPI DMA sur STM32 : Fonctions HAL

Pour la transmission

HAL_SPI_Transmit_DMA(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);

Après avoir appelé cette fonction, le périphérique SPI commencera à envoyer les octets de données présents dans le tampon, un par un, jusqu'à ce que la transmission soit terminée (en mode DMA).
Une fois la transmission terminée, la fonction ci-dessous sera appelée et exécutée.
Si vous souhaitez effectuer une action à la fin de la transmission, ajoutez-la dans votre code source de l'application (fichier main.c)

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX Done .. Do Something ...
}
```

Mode SPI DMA sur STM32 : Fonctions HAL

Pour la réception

```
HAL_SPI_Receive_DMA(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size);
```

Après avoir appelé cette fonction, le périphérique SPI commencera à recevoir les octets de données entrants dans le tampon, un par un, jusqu'à ce que la réception soit terminée (en mode DMA).
Une fois la réception terminée, la fonction ci-dessous sera appelée et exécutée.
Si vous souhaitez effectuer une action à la fin de la réception,
ajoutez-la dans votre code source de l'application (fichier main.c)

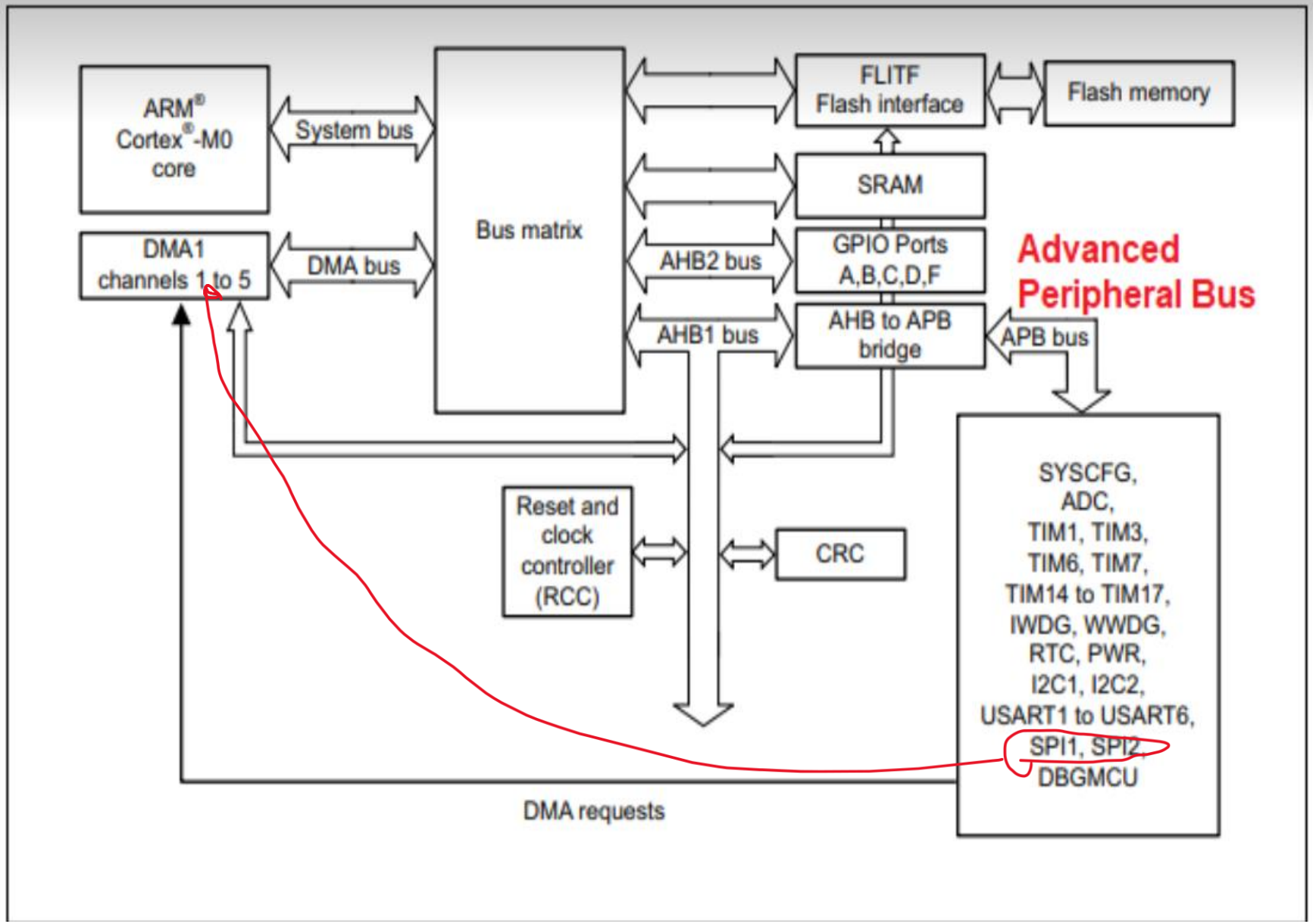
```
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // RX Done .. Do Something ...
}
```

Mode SPI DMA sur STM32 : Fonctions HAL

Pour la transmission-réception
HAL_SPI_TransmitReceive_DMA(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size);

Après avoir appelé cette fonction, le périphérique SPI commencera à transmettre et recevoir simultanément les octets de données présents dans les tampons, un par un, jusqu'à ce que l'opération soit terminée (en mode DMA).
Une fois la transmission-réception terminée,
la fonction ci-dessous sera appelée et exécutée. Si vous souhaitez effectuer une action à la fin de l'opération,
ajoutez-la dans votre code source de l'application (fichiermain.c)

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef * hspi)
{
    // TX-RX Done .. Do Something ...
}
```


ARM®
Cortex®-M0
core
System bus
DMA1
channels 1 to 5
DMA bus
Bus matrix
FLITF
Flash interface
Flash memory
SRAM
AHB2 bus
GPIO Ports
A,B,C,D,F
AHB1 bus
AHB to APB
bridge
APB bus
Advanced
Peripheral Bus
Reset and
clock
controller
(RCC)
CRC
SYSCFG,
ADC,
TIM1, TIM3,
TIM6, TIM7,
TIM14 to TIM17,
IWDG, WWDG,
RTC, PWR,
I2C1, I2C2,
USART1 to USART6,
SPI1, SPI2,
DBGMCU
DMA requests

Exemples d’utilisation du SPI dans NUCLEO-F446RE

tester le SPI en mode loopback:Le test SPI (Serial Peripheral Interface) en mode **loopback** est une méthode couramment utilisée pour vérifier le bon fonctionnement de l'interface SPI sans nécessiter de périphériques externes. Ce test permet de valider la communication entre les broches de transmission (MOSI) et de réception (MISO) en boucle fermée.

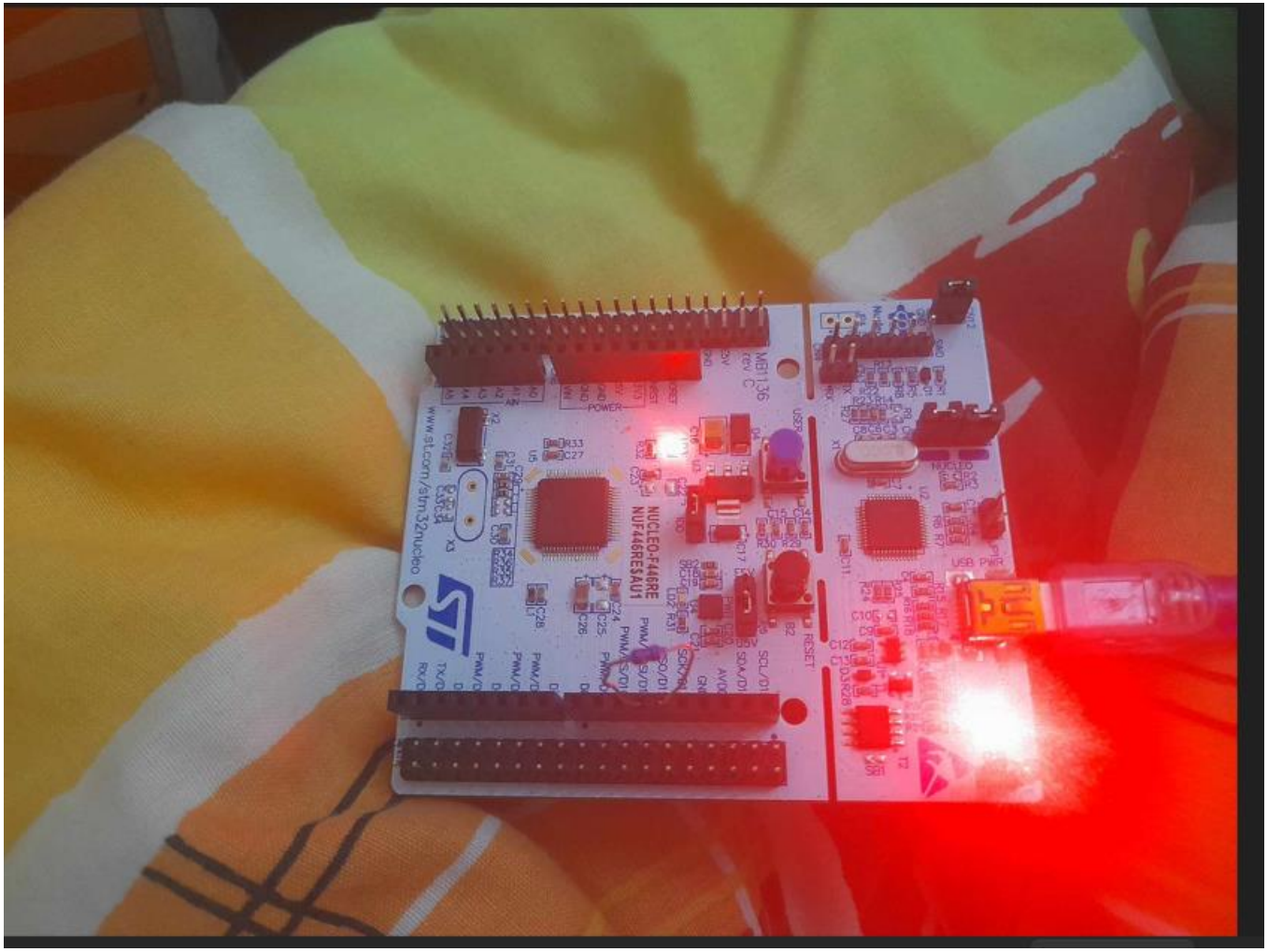
Validation des résultats :

•Comparer les données reçues avec celles envoyées.

•Si les données correspondent, le test est concluant et confirme que l'interface SPI fonctionne correctement.

•En cas de différence, cela peut indiquer un problème matériel ou une mauvaise configuration du SPI.

NUCLEO-F446RE



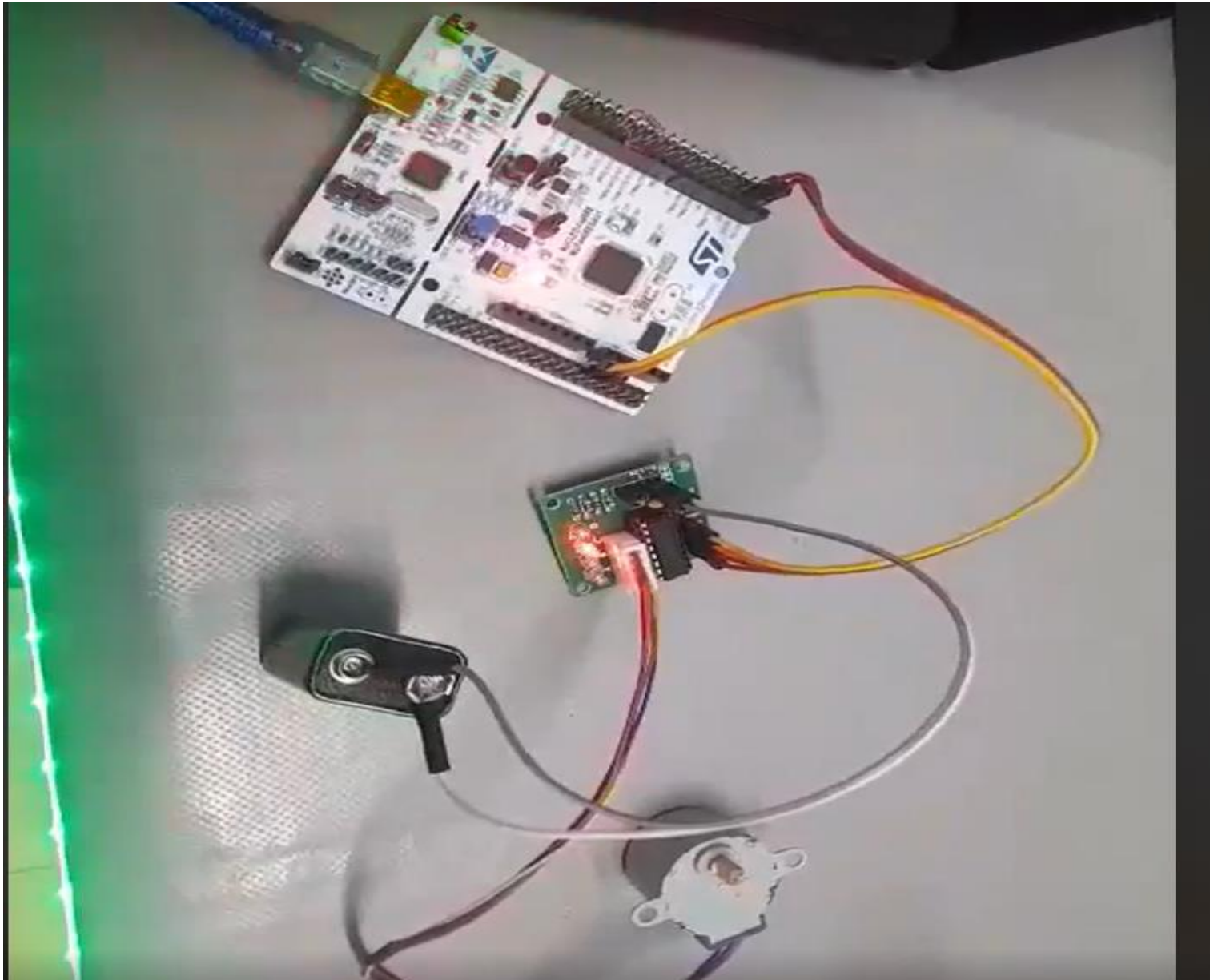
Code Source

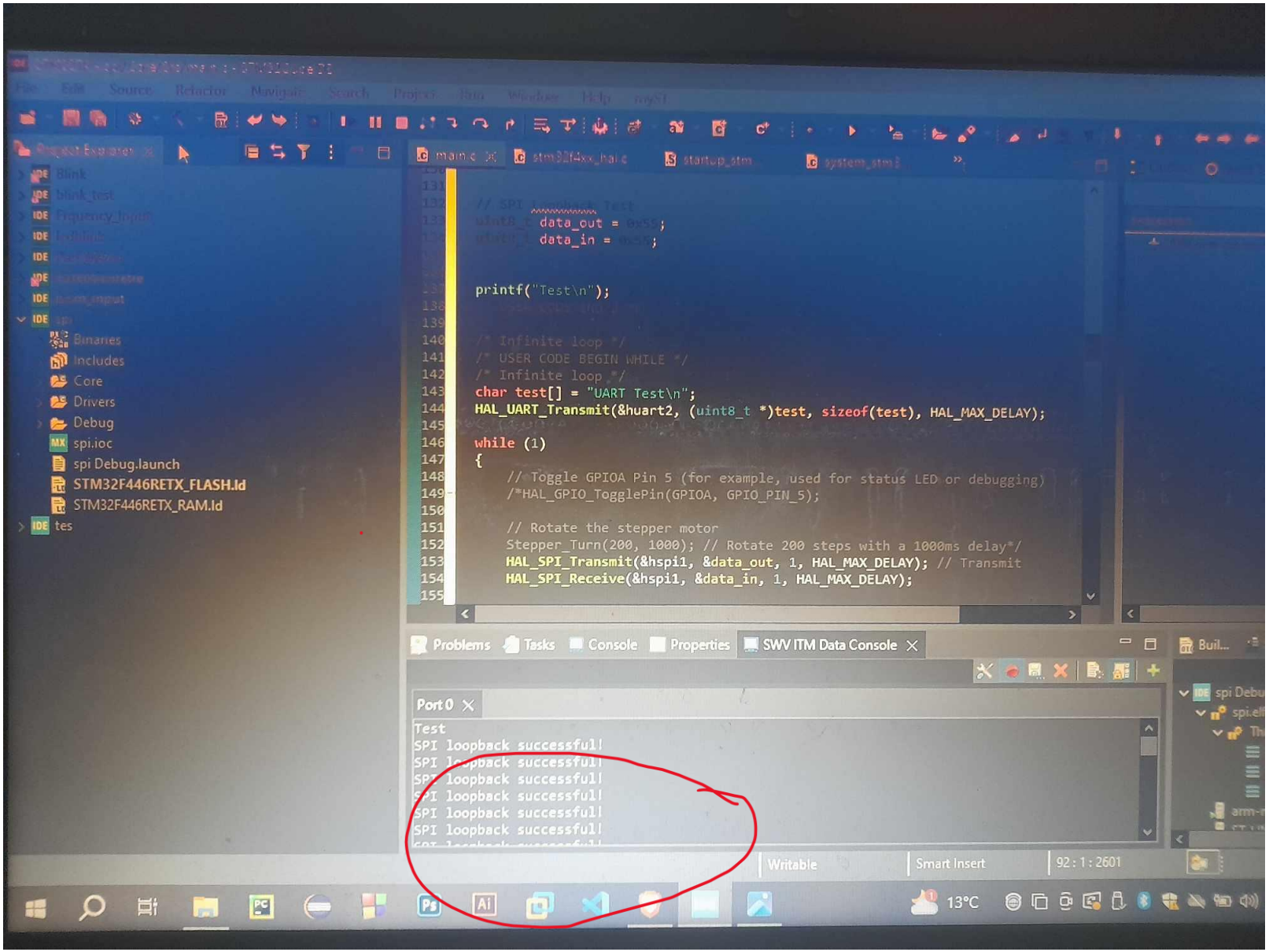
```
while (1)
{
    HAL_SPI_Transmit(&hspi1, &data_out, 1, HAL_MAX_DELAY); // Transmit
    HAL_SPI_Receive(&hspi1, &data_in, 1, HAL_MAX_DELAY);


    if (data_out == data_in) {
        printf("SPI loopback successful!\n");
    } else {
        printf("SPI loopback failed.\n");
    }

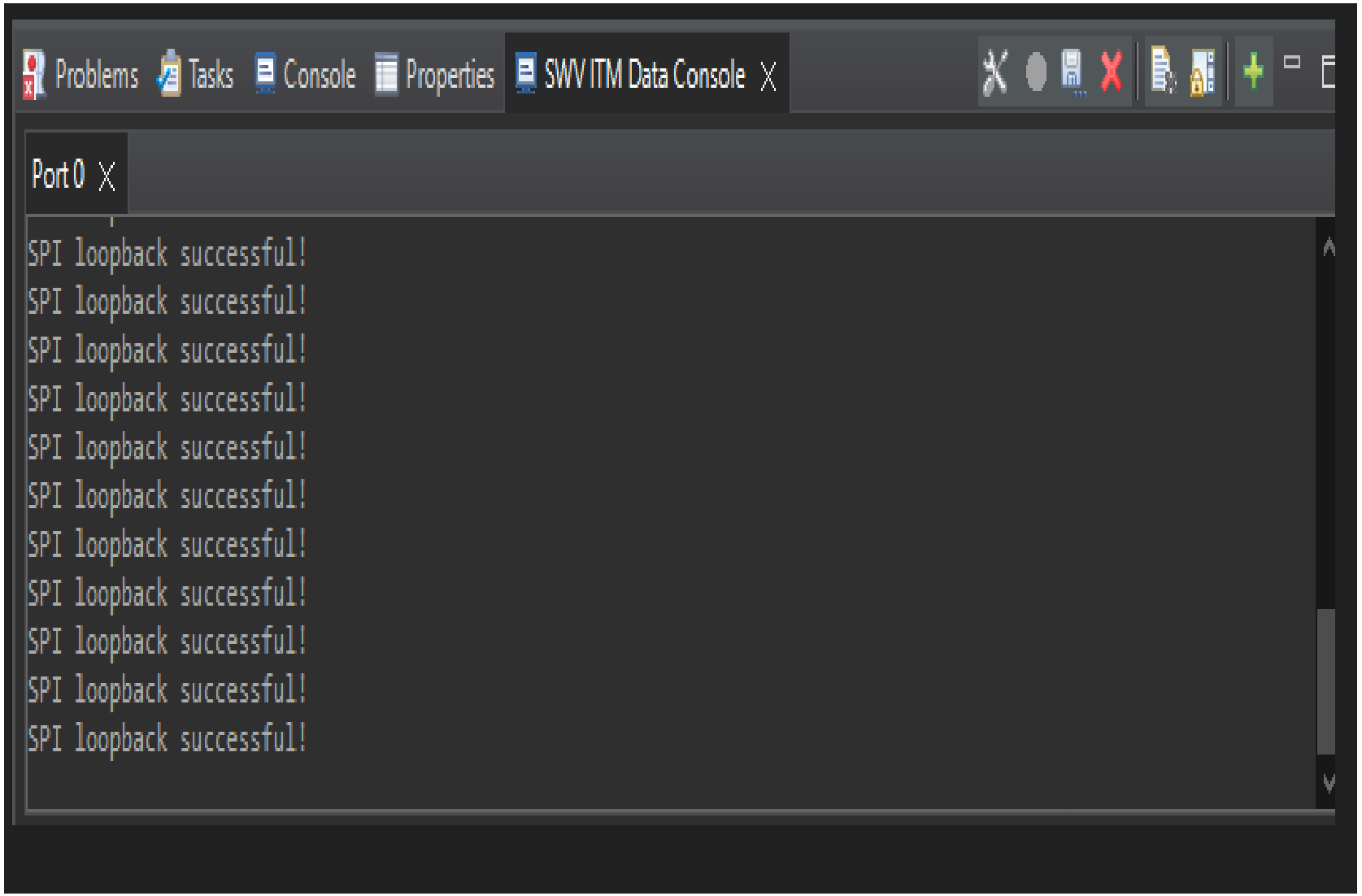
    // Add delay between iterations of SPI loopback test
    HAL_Delay(500);  // Add a 500ms delay to observe the SPI test output
}
```

Circuit pour le test SPI

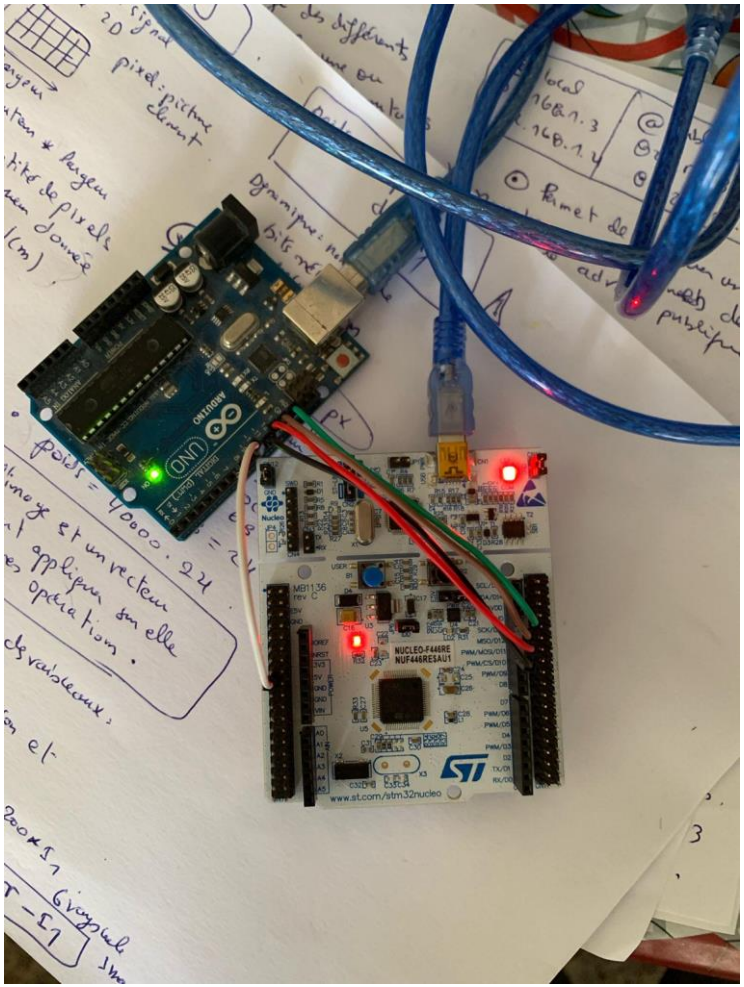


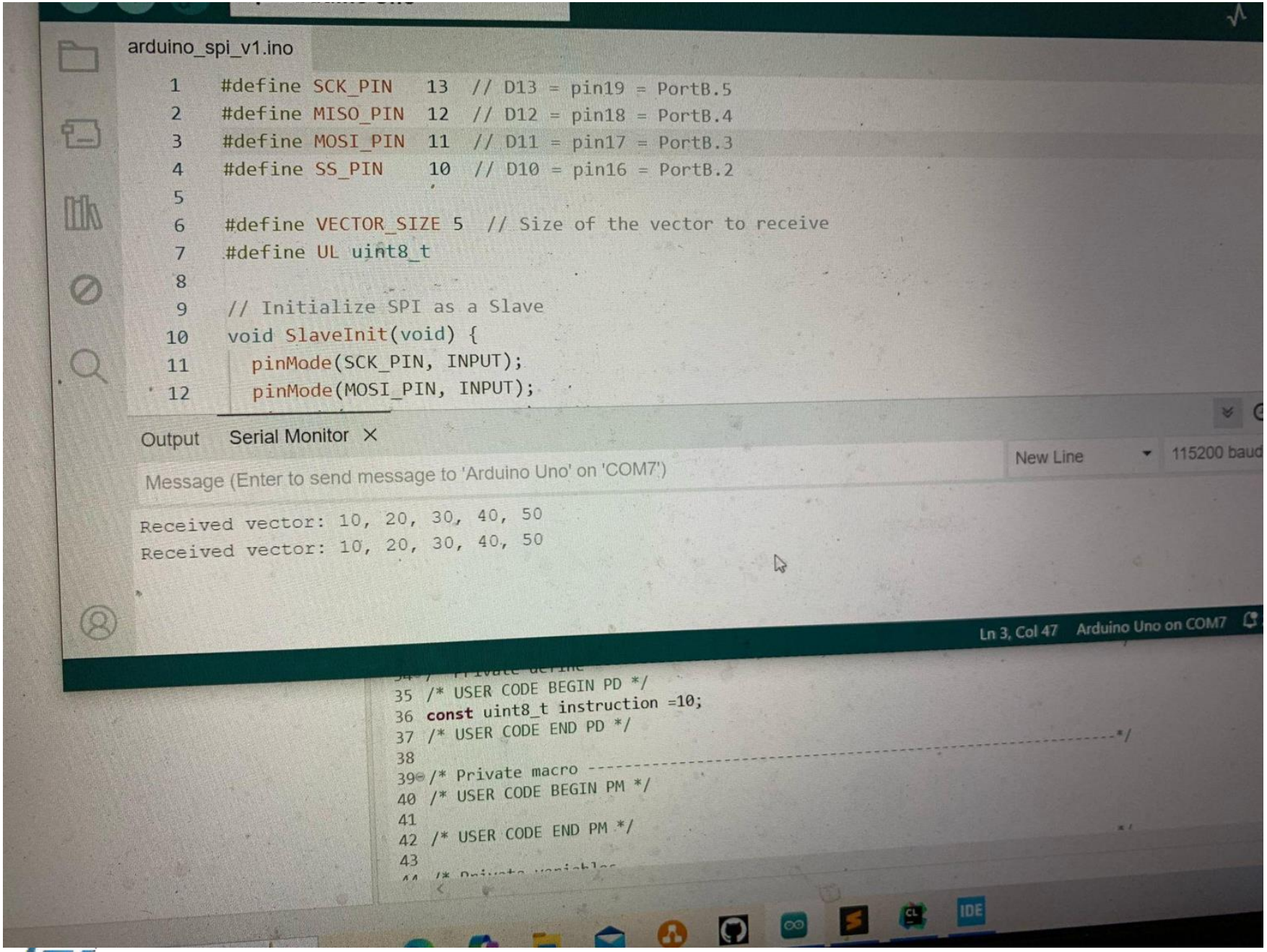

data_out = 0x55;
data_in = 0x00;
printf("Test\n");
/* Infinite loop */
/* USER CODE BEGIN WHILE */
/* Infinite loop */
char test[] = "UART Test\n";
HAL_UART_Transmit(&huart2, (uint8_t *)test, sizeof(test), HAL_MAX_DELAY);
while (1)
{
// Toggle GPIOA Pin 5 (for example, used for status LED or debugging)
/*HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
// Rotate the stepper motor
Stepper_Turn(200, 1000); // Rotate 200 steps with a 1000ms delay*/
HAL_SPI_Transmit(&hspi1, &data_out, 1, HAL_MAX_DELAY); // Transmit
HAL_SPI_Receive(&hspi1, &data_in, 1, HAL_MAX_DELAY);
Problems
Tasks
Console
Properties
SWV ITM Data Console
Port 0
Test
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
Writable
Smart Insert
92 : 1 : 2601
13°C

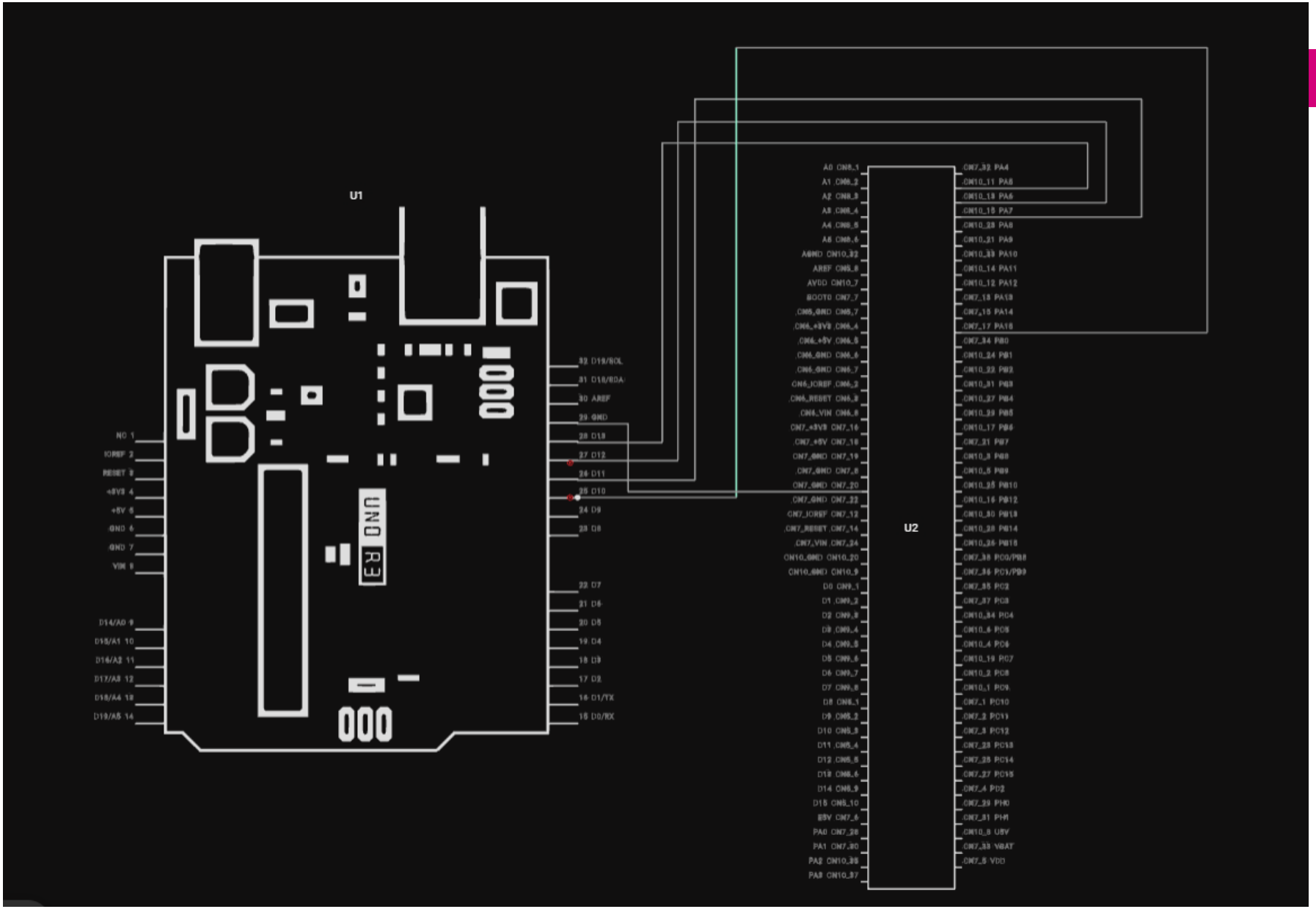
Résultat


Problems
Tasks
Console
Properties
SWV ITM Data Console
Port 0
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!
SPI loopback successful!

Exemple avec NUCLEO-F446RE(master) Arduino uno (slave)




arduino_spi_v1.ino
1 #define SCK_PIN 13 // D13 = pin19 = PortB.5
2 #define MISO_PIN 12 // D12 = pin18 = PortB.4
3 #define MOSI_PIN 11 // D11 = pin17 = PortB.3
4 #define SS_PIN 10 // D10 = pin16 = PortB.2
5
6 #define VECTOR_SIZE 5 // Size of the vector to receive
7 #define UL uint8_t
8
9 // Initialize SPI as a Slave
10 void SlaveInit(void) {
11 pinMode(SCK_PIN, INPUT);
12 pinMode(MOSI_PIN, INPUT);
Output
Serial Monitor
Message (Enter to send message to 'Arduino Uno' on 'COM7')
New Line
115200 baud
Received vector: 10, 20, 30, 40, 50
Received vector: 10, 20, 30, 40, 50
Ln 3, Col 47
Arduino Uno on COM7
35 /* USER CODE BEGIN PD */
36 const uint8_t instruction =10;
37 /* USER CODE END PD */
38
39 /* Private macro
40 /* USER CODE BEGIN PM */
41
42 /* USER CODE END PM */
43


U1
U2
UNO R3
NO 1
IOREF 2
RESET 3
+3V3 4
+5V 5
GND 6
GND 7
VIN 8
D14/A0 9
D15/A1 10
D16/A2 11
D17/A3 12
D18/A4 13
D19/A5 14
32 D19/SCL
31 D18/SDA
30 AREF
29 GND
28 D13
27 D12
26 D11
25 D10
24 D9
23 D8
22 D7
21 D6
20 D5
19 D4
18 D3
17 D2
16 D1/TX
15 D0/RX

Code Arduino 1

```
#define SCK_PIN   13  // D13 = pin19 = PortB.5
#define MISO_PIN  12  // D12 = pin18 = PortB.4
#define MOSI_PIN  11  // D11 = pin17 = PortB.3
#define SS_PIN    10  // D10 = pin16 = PortB.2

#define VECTOR_SIZE 5  // Size of the vector to receive
#define UL uint8_t

// Initialize SPI as a Slave
void SlaveInit(void) {
  pinMode(SCK_PIN, INPUT);
  pinMode(MOSI_PIN, INPUT);
  pinMode(MISO_PIN, OUTPUT);  // Set MISO output
  pinMode(SS_PIN, INPUT);

  // Enable SPI, set to Slave mode, and configure SPI mode
  SPCR = (1 << SPE) | (0 << DORD) | (0 << MSTR) | (0 << CPOL) | (0 << CPHA) | (0 << SPR1) | (1 << SPR0);
}

// Function to read a byte via SPI
uint8_t ReadByte(void) {
  while (!(SPSR & (1 << SPIF))) ;  // Wait for reception to complete
  return SPDR;                     // Return the received byte
}

void setup() {
  Serial.begin(115200);
  SlaveInit();  // Initialize SPI in Slave mode
  delay(10);
  Serial.println("********** SPI Slave Ready **********");
}
```

Activer W
Accédez aux

life.augmented

Code Arduino 2

```
33  void loop() {
34    uint8_t data_vector[VECTOR_SIZE];  // Buffer to store the received vector
35    uint8_t index = 0;
36
37    // Wait for Slave Select (SS) pin to go low, indicating start of communication
38    while (digitalRead(SS_PIN) == 1) {}
39
40    // Read VECTOR_SIZE bytes from SPI
41    while (index < VECTOR_SIZE) {
42      data_vector[index++] = ReadByte();
43    }
44
45    // Print the received vector (no reversal needed)
46    Serial.print("Received vector: ");
47    for (uint8_t i = 0; i < VECTOR_SIZE; i++) {
48      Serial.print(data_vector[i]);
49      if (i < VECTOR_SIZE - 1) {
50        Serial.print(", ");
51      }
52    }
53    Serial.println();
54
55    // Wait for Slave Select (SS) pin to go high, indicating end of communication
56    while (digitalRead(SS_PIN) == 0) {}
57  }
58
```

life.augmented

Code NUCELO-F446RE

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2024 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <stdint.h>

/* USER CODE END Includes */
```

life.augmented

Spi send vector function

```
65  HAL_StatusTypeDef SPI_SendVector(SPI_HandleTypeDef *hspi, GPIO_TypeDef *CS_GPIO_Port, uint16_t CS_Pin, uint8_t *data, uint16_t size)
66  {
67      HAL_StatusTypeDef status;
68
69      // LOW CS PIN pull
70      HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);
71
72      // Transmit spI FUNCTION
73      status = HAL_SPI_Transmit(hspi, data, size, HAL_MAX_DELAY);
74
75      // HIGH CS PIN pull
76      HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
77
78      return status;  // Return the transmission status
79  }
80
```

Toutes les configurations ont été faites par cubemx qui donne un fichier .ioc

life.augmented

LE CODE DE MAIN

```
87  int main(void)
88  {
89
90    /* USER CODE BEGIN 1 */
91    char uart_buf[50];
92    int uart_buf_len;
93    char spi_buf[20];
94    uint8_t data_vector[] = {10, 20, 30, 40, 50};
95
96
97    /* USER CODE END 1 */
98
99    /* MCU Configuration--------------------------------------------------------*/
100
101   /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
102   HAL_Init();
103
104   /* USER CODE BEGIN Init */
105
106   /* USER CODE END Init */
107
108   /* Configure the system clock */
109   SystemClock_Config();
110
111   /* USER CODE BEGIN SysInit */
112
113   /* USER CODE END SysInit */
114
115   /* Initialize all configured peripherals */
116   MX_GPIO_Init();
117   MX_USART2_UART_Init();
118   MX_SPI1_Init();
119   /* USER CODE BEGIN 2 */
120 //if we are going to put it in the main loop we should make intruptions handlers , since we will be overloading the arduino serial monitor
121   HAL_StatusTypeDef status = SPI_SendVector(&hspi1, GPIOA, GPIO_PIN_15, data_vector, sizeof(data_vector));
```

life.augmented

Bibliographie et sources

https://www.moussasoft.com/tutoriels-electroniques

https://www.st.com/en/microcontrollersmicroprocessors/stm32-32-bit-arm-cortex-mcus.html

www.espacetechnologue.com

Conclusion



Pour conclure e SPI est une solution de communication puissante et polyvalente, idéale pour les applications où la vitesse et la simplicité sont des priorités, mais il peut être moins adapté pour des systèmes avec un grand nombre de périphériques.